Trabalho — Programação Paralela

O objetivo deste projeto é implementar uma aplicação sequencial e transformá-la em duas versões paralelas. Cada grupo deverá desenvolver:

1. uma versão sequencial;
2. uma versão paralela utilizando múltiplos processos e memória compartilhada;
3. uma versão paralela utilizando múltiplas threads;
4. uso de containers como, ALTERNATIVA, para execução das versões paralelas;

Ao final, as três versões deverão ser comparadas experimentalmente.

O trabalho deverá ser realizado em grupos de no máximo 3 alunos. Todos os integrantes do grupo deverão participar efetivamente do desenvolvimento e deverão ser capazes de explicar INTEGRALMENTE a solução apresentada. A avaliação da apresentação será **individual**, mesmo que o desenvolvimento seja realizado em grupo.

Se preferirem, cada grupo pode escolher uma aplicação sequencial existente que será utilizada como base para o projeto ao invés de implementar a aplicação sequencial. A aplicação **poderá** ser:

- desenvolvida pelo próprio grupo anteriormente;
- uma aplicação disponibilizada como código aberto no git;
- outra aplicação, desde que previamente aprovada pelo professor.

A aplicação deverá possuir complexidade suficiente para permitir uma análise significativa de paralelização. Não serão aceitas aplicações extremamente simples cuja paralelização consista apenas em distribuir uma operação elementar entre trabalhadores independentes. Exemplos de aplicações **inadequadas**: simples soma de valores; cálculo de uma operação matemática simples; execução repetitiva de uma função sem compartilhamento ou coordenação; programas cuja paralelização não permita discutir sincronização, comunicação ou divisão de trabalho.

Não pode ter repetições, ou seja, dois ou mais grupos implementarem a mesma aplicação.

Prazo para DEFINIÇÃO dos grupos e proposta de aplicação: 11/09/26. A proposta deverá conter:

- descrição resumida do problema;
- linguagem de programação utilizada;
- origem da aplicação/código;
- descrição da entrada;
- descrição da saída;
- identificação das partes que poderão ser paralelizadas

A versão sequencial deverá possuir uma execução que leve **pelo menos 2 segundos** para uma entrada de teste. O tempo deverá ser medido em condições normais de execução,

utilizando uma entrada suficientemente grande para representar uma carga de trabalho realista. O grupo deverá apresentar a entrada utilizada e demonstrar o tempo de execução durante a apresentação. A aplicação deverá realizar **efetivamente** processamento computacional durante esse período.

As três versões (sequencial, com processos e com threads) deverão produzir resultados equivalentes para as mesmas entradas.

O número de processos/threads **não poderá estar fixo no código**. Deverá ser fornecido no momento da execução. O grupo deverá realizar uma análise comparativa entre as três implementações. Cada grupo deverá executar testes para obter o tempo de execução utilizando diferentes quantidades de processos e threads. Recomenda-se, no mínimo: 2, 4 e 8 processos e threads. Os valores deverão ser obtidos por meio de execuções reais da aplicação FORA do container.

O grupo deverá calcular o speedup das versões paralelas. O grupo deverá interpretar os resultados. Não basta apresentar os números.

Os resultados (tempo de execução e speedup) deverão ser apresentados graficamente.

No mínimo, deverá ser apresentado:

Gráfico 1 — Tempo de execução e **Gráfico 2** — Speedup

Comparação entre:

- sequencial;
- processos;
- threads.

Demonstrando o comportamento da aplicação com diferentes quantidades de processos e threads.

O grupo deverá disponibilizar sua aplicação em um container, contendo:

- código da aplicação;
- dependências;
- ambiente necessário para execução;
- configuração necessária;
- instruções de execução.

Deverá ser fornecido um: Dockerfile ou docker-compose.yml quando houver necessidade de múltiplos serviços. A aplicação deverá poder ser executada sem a necessidade de instalar manualmente suas dependências no sistema operacional do usuário.

As aplicações devem ser executadas dentro do container com diferentes limites de CPU. docker run –cpus=”1” ou docker run –cpus=”2” ou docker run –cpus=”4”.

Executar novamente as versões paralelas com 2, 4 e 8 processos/threads em containers com limite de CPU 1, 2 e 4.

Gráfico 3: Tempo de execução das versões paralelas FORA do container X Tempo de execução das versões paralelas DENTRO do container

Entrega 1 - Proposta - 11/09/2026

Entrega 2 — Implementação sequencial — (junto com a apresentação)

Entrega 3 — Implementações paralelas — (junto com a apresentação)

Entrega 4 - Container - Docker + limite de CPU + análise do comportamento. — (junto com a apresentação)

Entrega 5 — Apresentação — (data a ser definida)

A apresentação será realizada presencialmente.

O grupo deverá apresentar:

1. problema original;
2. funcionamento da versão sequencial;
3. estratégia de paralelização;
4. implementação com processos;
5. memória compartilhada;
6. sincronização;
7. implementação com threads;
8. resultados experimentais;
9. comparação entre as três versões;
10. conclusões.

Além da apresentação do grupo, **cada integrante será avaliado individualmente**.

A apresentação não será apenas uma exposição dos slides. Poderão ser realizadas perguntas individualmente a qualquer integrante sobre qualquer parte do trabalho. Embora o projeto seja realizado em grupo, a nota não será necessariamente igual para todos os integrantes. A avaliação **individual** considerará principalmente:

- conhecimento do código;
- compreensão da estratégia de paralelização;
- domínio dos conceitos utilizados;
- capacidade de explicar decisões técnicas;
- capacidade de interpretar os resultados;
- capacidade de responder às perguntas do professor.

Portanto, a avaliação será baseada não apenas no código entregue, mas principalmente na capacidade do aluno de compreender, explicar, executar e defender sua solução.

Uma boa solução não é necessariamente aquela que apresenta o menor tempo de execução. Uma solução paralela que seja mais lenta que a solução sequencial não será automaticamente considerada errada. Será valorizada uma aplicação que demonstre:

- boa identificação das oportunidades de paralelização;
- divisão adequada do trabalho;
- utilização correta de processos;
- utilização correta de threads;
- utilização real de memória compartilhada;
- tratamento adequado de regiões críticas;
- resultados corretos;
- experimentos bem planejados;
- análise crítica dos resultados;
- compreensão dos motivos pelos quais o desempenho melhora ou piora.

No dia da apresentação, cada grupo deverá enviar/compartilhar por email um zip contendo:

```
/projeto
   /sequencial
      código-fonte
      README
   /processos
      código-fonte
      README
   /threads
      código-fonte
      README
   /container
      Dockerfile
   /testes
      entradas
      resultados
   /documentacao
      apresentacao.pdf
```

O `README` deverá explicar como:

1. instalar dependências;
2. compilar o programa;
3. executar a versão sequencial;
4. executar a versão com processos;
5. executar a versão com threads.

Em caso de plágio da internet ou de algum outro trabalho a nota será desconsiderada; Todos os alunos precisam comparecer na aula, mesmo que seja apresentação de outro

grupo, para caso ocorram dúvidas ou problemas as mesmas sejam solucionadas. Vale PONTO EXTRA a participação durante a apresentação de outros trabalhos

Datas da apresentação: 29/10, **30/10**, 05/11, **06/11** e 12/11 e **13/11**

Pontuação: **10,0 (P1)**